Experiment No.: 02
Title: Implementation of Fenwick Tree

Batch: B3 **Roll No.: 16014225801** **Experiment No.: 2**

Aim: Implementation of Fenwick Tree

Resources needed: Access to Hackerearth platform

Theory:

Let $*$ be some binary operation defined on a set of elements S and let $(S,*)$ be a monoid. This means $*$ satisfies ***closure, associativity, identity*** properties:-

- Given $x, y \in S,\ x * y \in S$:- closure
- Given $x, y, z \in S,\ (x * y) * z = x * (y * z)$:- associativity
- There exists some $e \in S$ such that for all $x \in S,\ x * e = e * x = x$:- identity

Examples:-

1. $(Z, +)$:- the set of integers under addition operation
2. Consider the set of all possible strings S made up of symbols from some symbol set Σ Let xoy mean the concatenation of strings x and y. Then (S, o) is also a monoid. The empty string ϵ acts as an identity.

<u>Description of the task</u>:-

Consider an array A of length N containing elements of some monoid $(S, *)$. A ***prefix aggregation*** of length $1 \leq i \leq N$ is an operation $prefix(i) = A[1] * A[2] * \ \ldots * A[i]$.

An ***update*** is an operation where given $1 \leq i \leq N$ and given a value ***key*** , we set $A[i] = key$, replacing an old value.

We get a sequence of requests for many such update and prefix aggregations. We want to perform both of these operations in a way to reduce the cost of both the operations.

There are 2 natural options but both of the approaches have higher time complexity:-

- No extra memory. Update an element $A[i]$ has $O(1)$ time complexity. But to perform a prefix aggregation $prefix(i)$ is $O(i)$ operation which in the worst case is $O(N)$
- Use another array, say, ***prefix***$[1..N]$ where $prefix[k] = \sum_{i=1}^{k} A[i]$, for all $1 \leq k \leq N$.

Now $prefix(i)$ has $\Theta(1)$ time complexity but update element at k requires updating ***prefix***$[i]$ for all $k \leq i \leq N$ which is $O(N)$ operation in the worst case. Additionally, space complexity is $\Theta(N)$. Time complexity of initializing ***prefix*** array is $\Theta(N)$ using running prefix operations.

Fenwick tree is an approach somewhere in between the above two :- ***store partial prefix aggregations with minimal overlap and use the non-overlapping partial prefix aggregations to get a bigger aggregation***.

Fenwick Tree (also called a **Binary Indexed Tree (BIT)**) is really a fast indexing scheme for answering prefix aggregation queries. Let us take $k = 22$ to see how it works.

- Expand $k = 22$ in binary. $(22)_{10} = (10110)_2 = 16 + 4 + 2$

- $prefix(22)$ is asking for aggregate of block of size 22. If we have 3 disjoint aggregates of 16 consecutive elements (***named, say, prefixsize(16)***) , 4 consecutive elements(***named, say, prefixsize(4)***) and 2 consecutive elements (***named, say, prefixsize(2)***), all within the range $[1 \dots 22]$ then

$$prefix(22) = prefixsize(16) * prefixsize(4) * prefixsize(2)$$

because of associativity of the $*$ operation.

- Perform following steps to answer $prefix(22)$: Set $answer = 0$

1. Find out the least power of 2 contributing to the magnitude of k. Here, it is $2^1 = 2$. Update $answer = answer * prefix(22)$
2. Update as $k = k - 2$ i.e $k = 20$
3. Find out the least power of 2 contributing to the magnitude 20. Here, it is $2^2 = 4$. Update $answer = answer * prefix(20)$
4. Update as $k = k - 4$ i.e $k = 16$
5. Find out the least power of 2 contributing to the magnitude 16. Here, it is $2^4 = 16$. Update $answer = answer * prefix(16)$
6. Update as $k = k - 16$ i.e $k = 0$. STOP

Final value of $answer = prefix(22) = A[1] * A[2] * A[3] * \dots * A[22]$

Array $prefix[1..N]$ **stores precomputed partial aggregations** $prefix[i] = A[i] * A[i-1] * \dots * A[j]$, **for some** $1 \le j \le i$ **Calculate_Prefix(** k**)**		
Step	**Operation**	**Comment**
1	Set $answer = e$	Initialize the answer
2	while($k > 0$)	While some block remains
3	$answer = answer * prefix[k]$	Update answer to include aggregation of this particular block
4	$k = k - (k\&(-k))$	Go to the next partial block
Time Complexity:- $O(1)$ (***steps 3 and 4***) Overall :- $O(log(k))$ (***running over complete binary representation of*** k)		

The above works only when $prefix[1..N]$ is precomputed.

Array $prefix[1..N]$ **is cleared to the value** e *ie.* **for all** $1 \le i \le N \;\; prefix[i] = e$

Prepare_PartialPrefix( )		
Step	**Instruction**	**Comment**
1	for($i = 1;\ i \leq N;\ i++$)	For every index calculate the partial aggregation
2	$prefix[i] = e;$ $size = i\&(-i)$	Initialize aggregation Decide the block size
3	for($j = i;\ j > i - size;\ j--$)	
4	$prefix[i] = prefix[i] * A[j]$	Cumulatively update the aggregation
Time complexity:- $O(log(i))$ for each $1 \leq i \leq N$ (***steps 2 to 4***) Overall :- $\leq \sum_{i=1}^{N} log(i) \in O(N\ log(n))$ (***running 2 to 4 N times***)		

The update operation needs to be taken care of. Suppose we want to update the element at the index k so that $A[k] = old\ A[k]\ *\ \delta$ where δ is the change.

- Definitely $prefix[i]$ will not change for all $i < k$
- Find all the indices $i \geq k$ for which $prefix[i]$ needs to updated because $A[k]$ is the part of partial aggregation at $prefix[i]$.

Update(k)		
Step	**Instruction**	**Comment**
1	$A[k] = A[k] * \delta;$ $i = k$	Update $A[k]$ by δ
2	while($i \leq N$)	
3	$prefix[i] = prefix[i]\ *\ \delta;$	Update partial aggregation at the index i
4	$size = i\&(-i)$	What is the size of partial aggregation at i ?
5	$i = i + size$	Jump by $size$ to next index. That is the index which also contains $A[k]$
Time complexity:- $O(1)$ (***steps 3 to 5***) Overall :- $O(\ log(n))$ (***running 3 to 5 before high index crosses*** N)		

Procedure:

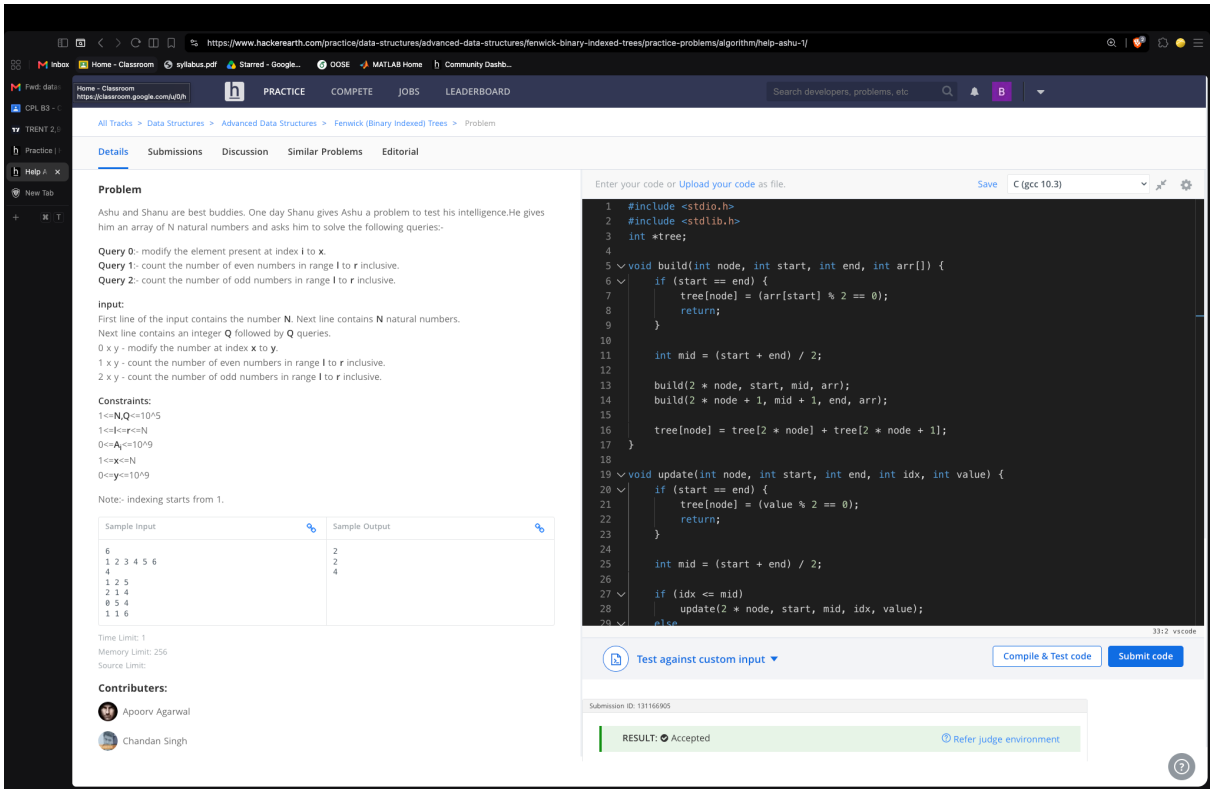
Write a C/C++ program to solve the following problem posted on Hackerearth. Ashu and Shanu are best buddies. One day Shanu gives Ashu a problem to test his intelligence.He gives him an array of N natural numbers and asks him to solve the following queries:-

Query 0:- modify the element present at index i to x.

Query 1:- count the number of even numbers in range l to r inclusive.

Query 2:- count the number of odd numbers in range l to r inclusive.

Results: (Steps with screenshots)



Code:

```
#include <stdio.h>
#include <stdlib.h>
int *tree;

void build(int node, int start, int end, int arr[]) {
    if (start == end) {
        tree[node] = (arr[start] % 2 == 0);
        return;
    }

    int mid = (start + end) / 2;
```

```
    build(2 * node, start, mid, arr);
    build(2 * node + 1, mid + 1, end, arr);

    tree[node] = tree[2 * node] + tree[2 * node + 1];
}

void update(int node, int start, int end, int idx, int value) {
    if (start == end) {
        tree[node] = (value % 2 == 0);
        return;
    }

    int mid = (start + end) / 2;

    if (idx <= mid)
        update(2 * node, start, mid, idx, value);
    else
        update(2 * node + 1, mid + 1, end, idx, value);

    tree[node] = tree[2 * node] + tree[2 * node + 1];
}

int query(int node, int start, int end, int l, int r) {
    if (r < start || end < l)
        return 0;

    if (l <= start && end <= r)
        return tree[node];

    int mid = (start + end) / 2;

    return query(2 * node, start, mid, l, r)
         + query(2 * node + 1, mid + 1, end, l, r);
}

int main() {
    int N;
    scanf("%d", &N);

    int *arr = (int *)malloc((N + 1) * sizeof(int));

    for (int i = 1; i <= N; i++)
        scanf("%d", &arr[i]);

    tree = (int *)malloc(4 * (N + 1) * sizeof(int));

    build(1, 1, N, arr);

    int Q;
    scanf("%d", &Q);

    while (Q--) {
        int type, x, y;
        scanf("%d %d %d", &type, &x, &y);

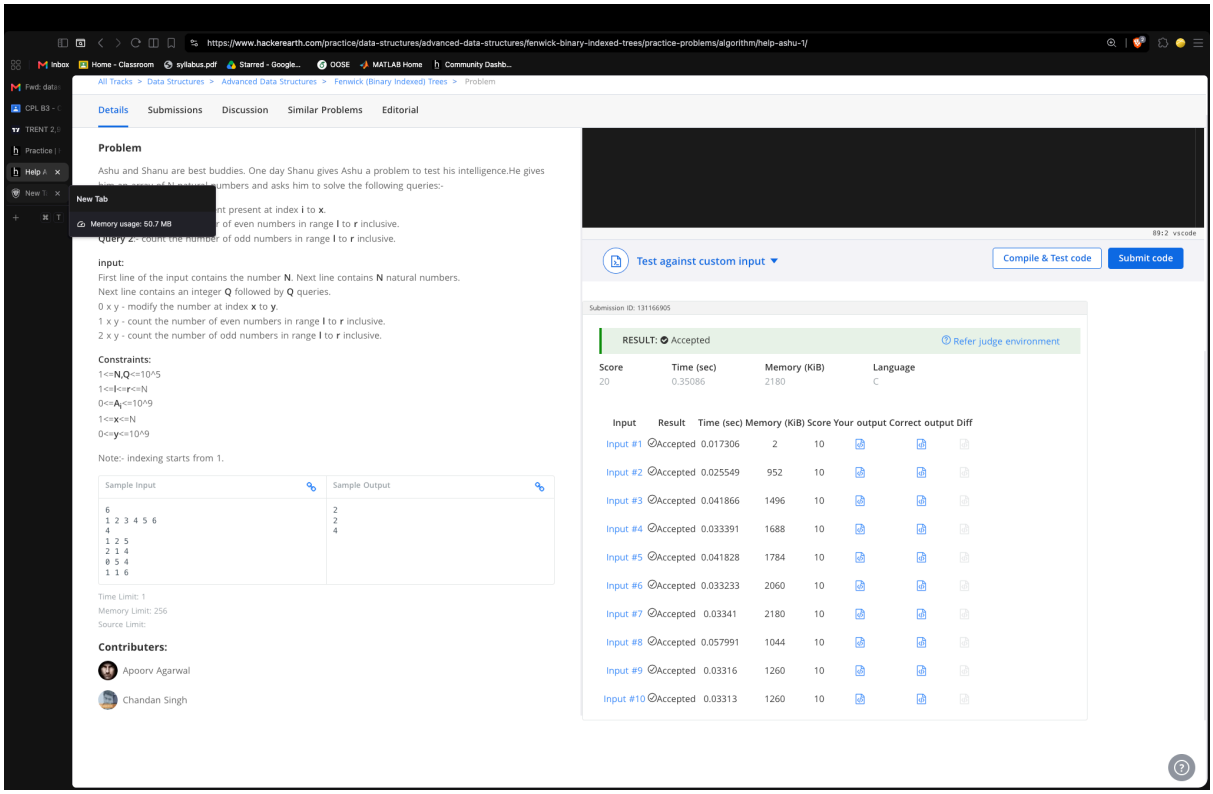
        if (type == 0) {
```

```
            // Modify index x to value y
            arr[x] = y;
            update(1, 1, N, x, y);
        }
        else if (type == 1) {
            // Count even numbers from x to y
            printf("%d\n", query(1, 1, N, x, y));
        }
        else if (type == 2) {
            // Count odd numbers from x to y
            int even = query(1, 1, N, x, y);
            int total = y - x + 1;
            printf("%d\n", total - even);
        }
    }

    free(arr);
    free(tree);

    return 0;
}
```

Output:



Part B – Use case implementation

Implementing a Fenwick Tree-based heatwave monitoring system that converts daily weather conditions into binary heatwave indicators. It efficiently supports updates and range queries,

enabling quick calculation of heatwave occurrences across selected dates with improved monitoring performance.

Code:

```
#include <iostream>
#include <vector>
using namespace std;

class FenwickTree {
private:
    vector<int> tree;
    int n;

public:
    FenwickTree(int n) {
        this->n = n;
        tree.resize(n + 1, 0);
    }

    // Add value to index
    void update(int index, int value) {
        while (index <= n) {
            tree[index] += value;
            index += index & (-index);
        }
    }

    // Prefix sum: 1 to index
    int query(int index) {
        int sum = 0;

        while (index > 0) {
            sum += tree[index];
            index -= index & (-index);
        }

        return sum;
    }

    // Range sum: l to r
    int rangeQuery(int l, int r) {
        if (l > r)
            return 0;

        return query(r) - query(l - 1);
    }
};

int main() {

    // Example:
    // 1 = heatwave day
    // 0 = normal day
```

```
    vector<int> heatwave = {
        0, 1, 1, 0, 1, 0, 1, 1, 0, 0
    };

    int n = heatwave.size();

    FenwickTree ft(n);

    // Build Fenwick Tree
    for (int i = 0; i < n; i++) {
        ft.update(i + 1, heatwave[i]);
    }

    // Query number of heatwave days
    // from day 2 to day 7
    cout << "Heatwave days from 2 to 7: "
         << ft.rangeQuery(2, 7) << endl;

    // Suppose day 5 changes from heatwave to normal
    // Original value = 1
    // New value = 0
    ft.update(5, -1);

    cout << "After updating day 5:" << endl;

    cout << "Heatwave days from 2 to 7: "
         << ft.rangeQuery(2, 7) << endl;

    return 0;
}
```

Output:

```
Heatwave days from 2 to 7: 5
After updating day 5:
Heatwave days from 2 to 7: 4
```

Post Lab Questions:

1. Can Fenwick tree be used to answer queries like finding the minimum number in the given array range $A[L \ldots R]$ of the array of numbers? If yes, how? If no, then why not?

Answer:

A standard Fenwick Tree cannot efficiently find the minimum value in an arbitrary range. Fenwick Trees are designed for associative operations with an inverse, such as sum. Minimum does not have an inverse operation, so

`min(prefix(r)) -min(prefix(l-1))`

is not valid. For range minimum queries, a Segment Tree is more suitable and supports both queries and updates in O(log N).

Conclusion:

Fenwick Trees efficiently handle cumulative frequency and range-sum queries with updates. Their limitations make Segment Trees preferable for minimum queries, demonstrating the importance of selecting appropriate data structures for specific operations.

Grade: AA / AB / BB / BC / CC / CD /DD

Signature of faculty in-charge with date

References:

1. Antti Laaksonen, “Guide to Competitive Programming”,Springer,2018
2. Gayle Laakmann McDowell,” Cracking the Coding Interview”,CareerCup LLC,2015
3. Steven S. Skiena Miguel A. Revilla,”Programming challenges, The Programming Contest Training Manual”, Springer, 2006
4. Antti Laaksonen, “Competitive Programmer’s Handbook”, Hand book, 2018
5. Steven Halim and Felix Halim, “Competitive Programming 3: The Lower Bounds of Programming Contests”, Handbook for ACM ICPC